COLEÇÃO TÉCNICA INTUIÇÃO

Luigi Scofano de Araujo

Arquitetura e implementação

Como executar entender e ampliar o código de Intuição

Este manual explica como executar, entender e ampliar a implementação atual de Intuição sem misturar apresentação, regras, dados e registros salvos. A aplicação é estática, usa módulos JavaScript nativos e não depende de um backend para jogar. A organização é híbrida: os primeiros jogos compartilham interfaces maiores; os posteriores têm diretórios próprios, rotas assíncronas e serviços de análise.

Base documental. Código de leitura congelado no commit 2354a3cd4d0a06ed66b665ca853a635b64e3a0a7. Os 11 jogos ativos são a referência. Propostas de organização e a preparação local para GitHub são identificadas como trabalho separado, não como mudanças publicadas.

Mapa de leitura

Seção	Uso principal
2 Execução local	Abrir o checkout correto sem compilação
3 Fluxo da aplicação	Entender a ligação entre entrada e resultado
4 Organização do código	Localizar motores, UI, dados e serviços
5 Contratos das regras	Preservar estados e transições determinísticas
6 Rotas e interação	Navegação, carregamento e ciclo de vida
7 Persistência	Identidade, histórico e migrações
8 Importação	Limites e validação de JSON
9 Geração e análise	Certificados, workers e cancelamento
10 Recursos visuais	Arte sem acoplamento indevido às regras
11 Extensão segura	Roteiro para conteúdo e novos módulos
12 Referências	Fontes e fronteiras da documentação

2 Execução local

O diretório **dist** contém a aplicação executável, incluindo HTML, CSS, módulos, dados e recursos. Neste baseline não existe package.json na raiz nem etapa obrigatória de npm install ou npm build. O nome dist não significa que seja necessário recompilar um projeto-fonte externo.

```
python -m http.server 8766 --bind 127.0.0.1 --directory dist
# Abra http://127.0.0.1:8766/
```

Execute o comando na raiz do checkout. No Windows, o lançador py pode substituir python. O servidor apenas entrega arquivos; não processa partidas. Use HTTP local em vez de abrir index.html por file://, pois o carregamento de módulos pode ser bloqueado pelo navegador.

Verificação inicial

• Confirme que index.html abre e que as três categorias levam ao catálogo de 11 jogos. A seleção de cada puzzle deve listar 80 fases; Versus apresenta mapas, não uma campanha numerada.

• Abra uma fase, faça uma alteração, volte à seleção e retome. Recarregue apenas depois de verificar o aviso de armazenamento. Isso testa navegação e persistência sem apagar o perfil.

• Em uma cópia de trabalho, execute os testes Node aplicáveis. Node 24.22.1 foi o ambiente da auditoria registrada; isso é uma versão observada, não uma matriz formal de compatibilidade.

Dependências reais do ambiente

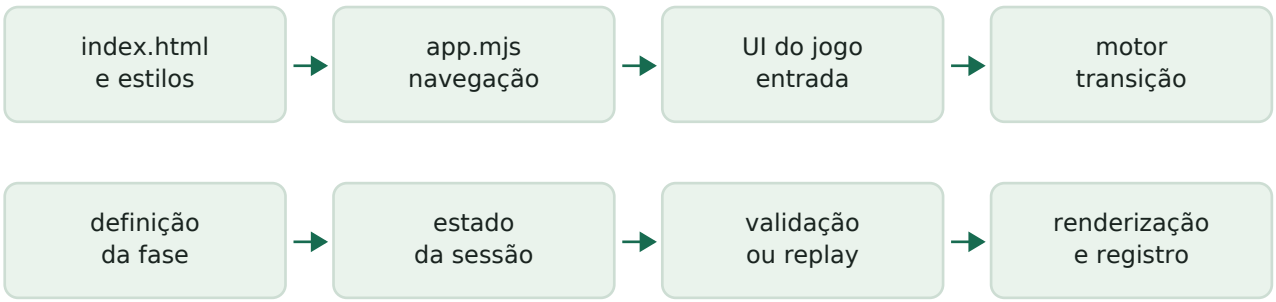
O navegador fornece ES modules, SVG, armazenamento local e, quando disponível, Web Workers. Python é somente uma opção de servidor estático. Node executa os testes e scripts de autoria, não é requisito para o usuário jogar. Módulos antigos e ferramentas de construção podem ter necessidades próprias; consulte o script antes de executá-lo.

> Uma cópia baixada não se atualiza sozinha. O armazenamento pertence à origem do navegador: mudar de host, porta, perfil ou aparelho pode mostrar outro progresso. Ausência de progresso em uma origem nova não prova que o registro antigo foi apagado.

Fontes desta seção: dist/README.md, dist/index.html, inventário do checkout e logs da auditoria do baseline.

3 Fluxo da aplicação

O ponto de entrada HTML carrega a aplicação e os estilos. A navegação por categoria controla quais cartões aparecem; cada módulo de jogo conserva a responsabilidade por abrir, esconder, salvar e restaurar sua própria tela. O URL registra a categoria ou o jogo, mas não substitui a definição da fase nem o save.



Duas famílias de integração

Reflexos e Corte certo passam pelo app.mjs, com geometria, respostas e geradores separados. **ELO, PRUMO, PACTO e DESVIO** passam por collection-ui.mjs e auxiliares compartilhados. **SINTONIA, REPASSE, ÓRBITA, BROTO e CERCO** têm rotas e interfaces dedicadas.

Não há uma interface universal única de motor. Geometria e símbolos validam uma posição; os jogos de comandos reproduzem sequências; os duelos produzem um novo estado após um lance. Um adaptador deve respeitar essas diferenças em vez de converter todos os jogos para uma regra artificial comum.

Separação recomendada durante manutenção

Camada	Pode decidir	Não deve decidir
Motor	Legalidade, transição e vitória	CSS, foco, localStorage
Dados	Tabuleiro, pistas, metas e identidade	Manipulação de DOM
UI	Gestos, mensagens e controles	Inventar uma vitória não validada
Serviço	Trabalho assíncrono e cancelamento	Confiar cegamente no retorno do worker
Persistência	Guardar e restaurar o estado válido	Promover posição de seleção a identidade

Fontes desta seção: dist/app.mjs, collection-ui.mjs, category-navigation.mjs e diretórios dos cinco jogos dedicados.

4 Organização do código

Área	Arquivos de entrada	Responsabilidade
Aplicação	dist/index.html e app.mjs	Casca, jogos originais e integração
Óptica	light-world.mjs e light-world-render.mjs	Traçado e desenho dos raios
Corte	curves.mjs, math.mjs e cut-fauna-geometry.mjs	Curvas, proporções e silhuetas
Grades lógicas	geometry-games.mjs, elo-rules.mjs e symbol-games.mjs	ELO, PRUMO e PACTO
Coleção	collection-ui.mjs e collection-state.mjs	UI, histórico e sessões compartilhadas
Dados ativos	levels.mjs, geometry-levels.mjs e symbol-levels.mjs	Entradas para catálogos materializados
Jogos dedicados	sintonia/, repasse/, orbita/, broto/, cerco/	engine, UI, route e auxiliares
Interação	direct-input.mjs e direct-actions.mjs	Gestos, teclado e transações
Apresentação comum	rules-preview, result-motion, game-victory	Regras, resultado e movimento
Evidência	docs/ e tests/	Relatórios, testes e autoria offline

Como localizar uma regra

Comece pela função exportada do motor e pelo teste correspondente. Depois siga o caminho da UI que chama essa função. Para um defeito visual, faça o inverso: localize a marcação e o CSS, confirme os dados recebidos e só então investigue o motor. Arquivos extensos de campanha são principalmente dados; tamanho em bytes não equivale a complexidade algorítmica.

Dívida técnica que merece cuidado

app.mjs e collection-ui.mjs concentram muitas responsabilidades; várias interfaces possuem linhas muito longas. Há um ciclo entre curves.mjs e cut-fauna-geometry.mjs. Extrair funções puras e formatar código são melhorias plausíveis, mas não estão automaticamente implantadas por esta documentação. Preserve os testes e as assinaturas públicas antes de reorganizar importações.

Fontes desta seção: Inventário de módulos e importações do baseline; dist/curves.mjs e cut-fauna-geometry.mjs.

5 Contratos das regras

A unidade de verificação deve ser a regra do jogo, não uma imagem ou um rótulo. Um estado aparentemente correto pode conter turno, histórico, reservas ou parâmetros inconsistentes. Os motores de BROTO e CERCO congelam estados retornados e reconstroem dados derivados durante a restauração.

Família	Operação principal	Invariante central
Reflexos	traceLightWorld	Raios obedecem reflexão, colisões e portais
Corte certo	basicCutState e curvedCut	Área útil e tolerância de ±3 pp
ELO e PRUMO	validateGeometry	Conectividade e regras específicas completas
PACTO	validateSymbols	Cotas e um parceiro ortogonal por casa
SINTONIA	replay	O comando comum move os painéis legalmente
REPASSE	replay	Transferências exatas e metas finais
ÓRBITA	replay	Permutações legais nos anéis
BROTO e CERCO	apply e restore	Transação válida e replay coerente

Exemplo de transação protegida

```
import {createCerco, applyCercoMove}
from './dist/cerco/engine.mjs';
const before = createCerco(0, 'classic');
const result = applyCercoMove(before, {from: 2, to: 7}, {
expectedPlayer: before.player, expectedTurn: before.turn
});
// result.state é novo; a origem passa a barreira.
```

O guard de turno impede que uma resposta atrasada execute um lance para outro jogador. A UI precisa validar que o pedido ainda pertence à tela ativa e então aplicar a transação. Não escreva diretamente no tabuleiro salvo para simular uma jogada.

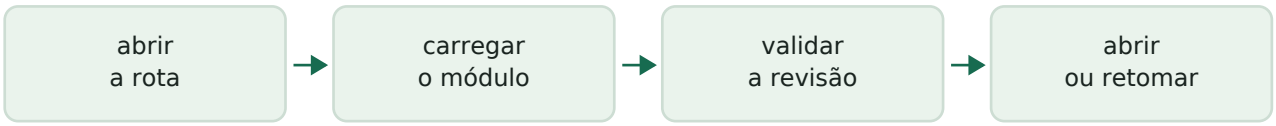
Em CERCO, bloquear uma peça isolada não encerra a partida se outra peça da equipe seguinte ainda pode mover. Em BROTO, todos os receptores são descobertos antes de marcar e remover flores; resolver cada raio removendo peças imediatamente mudaria a regra.

Fontes desta seção: dist/cerco/engine.mjs, broto/engine.mjs e motores listados na tabela.

6 Rotas e interação

As rotas dedicadas importam a UI sob demanda. Mantêm uma promessa de carregamento, um contador de revisão e uma indicação de visibilidade. Ao sair, a revisão muda; a resposta de um carregamento antigo não deve reabrir uma tela que o jogador já abandonou. Falhas de importação oferecem nova tentativa sem declarar perda de progresso.

Ciclo de vida que deve permanecer íntegro



• Ao esconder: encerrar gestos, invalidar buscas/importações pendentes, limpar timers e resultados temporários, persistir o que pertence à sessão e atualizar aria-pressed.

• Ao restaurar: ler somente o espaço do jogo, validar o conteúdo e conferir modo, identidade e versão. Um erro em ÓRBITA não deve impedir os demais jogos de abrir.

• Ao editar: registrar uma ação coerente no histórico. Um arraste contínuo não deve gerar centenas de passos de desfazer. Campo de texto JSON conserva seu desfazer nativo.

Não padronizar resultados incorretamente

Reflexos, Corte certo, ELO e PRUMO mantêm confirmação explícita; DESVIO confirma o lance. PACTO detecta automaticamente um tabuleiro completo e válido, além de manter o botão de confirmação para avaliar tentativas. SINTONIA, REPASSE e ÓRBITA também registram conclusão automaticamente. O código ativo deve prevalecer sobre testes históricos que descrevem um comportamento anterior. Uma frase histórica do README dizendo que todos sempre confirmam não deve governar o comportamento atual.

Acessibilidade e dispositivos

Teclado, controles numéricos e alternativas ao arraste precisam ser preservados onde implementados. Regras e estados usam marcação semântica e anúncios. CSS de movimento reduzido limita efeitos. Isso é implementação observada, não certificação integral de acessibilidade; foco, leitura de tela, toque e rolagem exigem inspeção interativa.

Fontes desta seção: dist/sintonia/route.mjs, orbita/route.mjs, direct-input.mjs, app.mjs; tests/automatic-completion-no-confirm.test.mjs.

7 Persistência e identidade

O progresso é local ao navegador. Não há conta nem sincronização prometida. Sessão, conclusão e assistência são conceitos diferentes: reiniciar o tabuleiro não deveria apagar o registro de vitória nem reverter o uso de um gabarito.

Registro	Exemplo de chave	Papel
Coleção	intuicao.collection.v1	Sessões, conclusões e eventos
Assistência	intuicao.answers.v1	Falhas e uso de resposta por identidade
Reflexos	intuicao.campaign.light.progress.v1	IDs concluídos e última fase
Corte	intuicao.campaign.cut.progress.v1	Progresso separado da campanha
Modo livre original	intuicao.endless.original.v1	Seed, opções e estado de Reflexos/Corte
Jogos dedicados	intuicao.sintonia.v1 e equivalentes	Campanha, preferências e sessão própria

Número de exibição não é identidade

Uma fase reaproveitada fisicamente pode manter seu ID apesar de mudar de posição. Alterar pistas, regras ou geometria requer identidade/versionamento coerentes e uma migração explícita. Não copie a vitória da antiga fase 16 para a nova fase 16 só porque ambas ocupam a mesma posição.

A coleção registra transações por cópias de estado. O histórico geral limita o passado a 300 entradas; ao persistir, a UI mantém até 300 para ELO e até 50 nos demais jogos dessa coleção. Os eventos locais são truncados aos 500 mais recentes. Esses limites não são quotas de partidas de toda a aplicação.

Restauração é validação

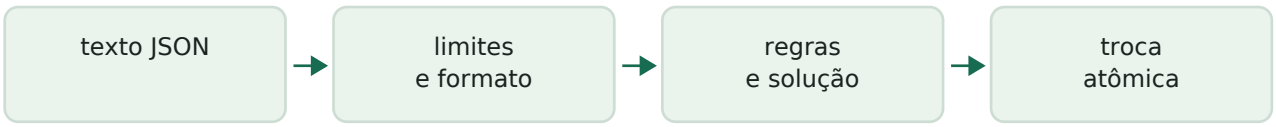
BROTO e CERCO reproduzem lances e comparam campos derivados, incluindo o ramo de refazer. O progresso de Reflexos não sobrescreve um registro bruto malformado como se fosse normal. Algumas rotinas toleram falha de armazenamento e exibem avisos; não se deve prometer recuperação se o navegador recusou a gravação.

Defeito conhecido do baseline. Na migração de um save histórico de SINTONIA, a normalização pode duplicar referências arquivadas na segunda passagem. O teste sintético observado passou de 482 para 964 referências, estabilizando na terceira. Não se observou alteração de movimentos ou conclusões nesse caso; isso não dispensa a correção de idempotência. A correção local 2dea81c passou os testes focados, mas não está implantada neste baseline.

Fontes desta seção: dist/collection-state.mjs, collection-ui.mjs, answer-progress.mjs, light-campaign-progress.mjs, cut-campaign-progress.mjs e motores de duelo.

8 Importação de níveis

A importação é uma fronteira de dados não confiáveis, não um mecanismo de carregar código. O fluxo antigo compartilhado aceita Reflexos, Corte, ELO, PRUMO, PACTO e DESVIO. Jogos dedicados têm contratos próprios; não suponha que o mesmo exemplo JSON sirva para os 11 jogos.



Limite comum	Valor
Tamanho codificado	200.000 bytes
Profundidade	16
Valores percorridos	10.000
Elementos por array	4.096
Campos por objeto	512
Comprimento de texto ou chave	4.096 caracteres

parseLevelJSON exige um objeto simples e game igual ao jogo selecionado. Bloqueia chaves como __proto__, constructor e prototype; recusa números não finitos, acessores, referências circulares e arrays esparsos em entradas programáticas. JSON.parse já elimina muitas formas de execução, mas os limites também protegem o custo de validar e desenhar.

Certificados e limites por jogo

Reflexos exige ângulos de uma solução e reproduz o traçado, sem provar unicidade. Corte verifica proporção no eixo básico e pode buscar um deslocamento sob orçamento. ELO e PRUMO recalculam solução única, com grades personalizadas máximas de 7×7 e 6×6; não herdam automaticamente o limite maior da campanha. PACTO exige 8×8. DESVIO permite até 16 casas e informa análise do jogo perfeito.

Zero soluções, múltiplas soluções e busca inconclusiva são resultados diferentes. Na coleção, o worker de importação tem limite documentado de 1,5 s; caminhos sem worker restringem ainda mais alguns casos. Cancelamento ou erro deve preservar a partida anterior. Campos de progresso no arquivo não autorizam concluir campanhas.

Fontes desta seção: dist/level-json-common.mjs, original-json.mjs, collection-json.mjs e IMPORTAR-NIVEIS-JSON.md.

9 Geração e análise assíncrona

Campanha e geração livre não são a mesma fonte de conteúdo. As campanhas atuais são catálogos materializados de 80 definições por puzzle. Scripts de autoria podem compor, selecionar e certificar dados offline. O modo livre gera ou seleciona desafios com seeds e parâmetros que precisam ser persistidos.

Responsabilidade dos serviços

O serviço de REPASSE separa canais de geração e análise. Cada canal possui serial, tarefa pendente e timeout; cancela a anterior antes de iniciar outra. Respostas são certificadas novamente. Ao falhar, a geração pode usar uma reserva certificada; análise de contingência reduz a busca em vez de inventar uma conclusão. O timeout padrão observado é 18 s, não uma garantia de latência em todo aparelho.

CERCO mantém uma busca por vez. Cancelar termina o worker, aborta o caminho cooperativo e rejeita a promessa pendente. Se Worker não estiver disponível, o bot pode usar o caminho assíncrono alternativo. A identidade do pedido protege contra resultados atrasados.

Reprodução útil de um incidente

Guardar	Por quê
Commit e versão de regras	O mesmo comando pode ter significado diferente em outra versão
Definição completa ou hash	Seed sozinha pode mudar com uma versão do gerador
Seed e opções normalizadas	Permitem tentar reproduzir a composição
Estado e sequência aplicada	Distingue falha de geração de falha de interação
Resultado e status da busca	Timeout não equivale a impossível
Origem da reserva quando usada	Explica o caminho de contingência sem mascará-lo

O que um certificado garante

Replay vencedor demonstra que existe aquela solução. Só uma busca completa com o predicado adequado sustenta unicidade, impossibilidade ou necessidade causal. Uma busca limitada que não encontrou resposta apenas informa que o orçamento terminou. Catálogo finito e controle de recência também não provam variedade infinita.

Fontes desta seção: dist/repasse/service.mjs e certification.mjs, cerco/service.mjs, sintonia/service.mjs e orbita/service.mjs.

10 Recursos e apresentação

O sistema visual combina SVG, imagens de fauna, ícones, CSS e metadados de apresentação. As imagens ajudam a identificar jogos e personagens, mas não devem alterar silenciosamente uma colisão, área geométrica ou regra de pareamento. Corte certo é uma exceção que exige ligação explícita entre a silhueta visual e a geometria jogável.

Componente	Ponto de entrada	Cuidado
Fauna compartilhada	fauna-art.mjs e game-nav-icons.mjs	Separar chave semântica de caminho do arquivo
SINTONIA	animal-roster.mjs, assets.mjs e scenery.mjs	Não trocar papel lógico por aparência
Acessórios	accessory-art.mjs e accessory-palette.mjs	Legibilidade de botão, cancela e destino
Corte fauna	cut-fauna-geometry.mjs e dados de silhueta	Preservar área útil e recortes
Resultados	result-motion.mjs e versus-victory.mjs	Não duplicar conclusão ao redesenhar
Fichas informativas	fauna-info e fauna-culture	Fontes factuais não são licenças de imagem

Procedimento para substituir um recurso

• Identifique todas as referências ao arquivo, dimensões, transparência, escala e recortes. Confira uso em cartão, tabuleiro, vitória e ficha, não apenas em uma tela.

• Mantenha a chave de domínio se o personagem continua sendo o mesmo. Se mudar geometria jogável, trate como alteração de conteúdo, não como simples troca de PNG.

• Registre origem, autor, licença ou autorização, transformação e restrições. Uma imagem presente no repositório não é, por esse fato, livre para redistribuição.

• Reveja contraste, reconhecimento sem cor e tamanhos pequenos; teste nomes longos e estados selecionado, bloqueado e concluído.

O inventário de recursos e proveniência desta coleção detalha os arquivos. Este manual não concede licença ao código, às imagens ou às fontes. A preparação local para GitHub não deve receber uma licença ampla por conveniência sem decisão do titular.

Fontes desta seção: dist/fauna-art.mjs, game-nav-icons.mjs, sintonia/assets.mjs, sintonia/animal-roster.mjs e módulos de arte citados.

11 Extensão segura

Adicionar uma fase ao formato existente

Primeiro descreva a descoberta que a fase exige. Construa a definição e uma solução de referência, execute o motor e registre por que as pistas ou obstáculos têm função. Só depois incorpore ao catálogo. As campanhas atuais têm contrato de 80 fases; uma adição não deve aumentar esse total silenciosamente. Uma substituição exige decisão editorial e migração de identidade.

Passo	Saída verificável
1 Definição	Dados normalizados e versão de regras
2 Certificação	Replay, solução completa e limites declarados
3 Curadoria	Faixa, ideia principal, alternativas e comparação com vizinhas
4 Identidade	ID estável ou novo ID justificado
5 Integração	Seleção, retorno, vitória e modo correto
6 Migração	Fixtures antigas preservadas sem vitória por posição
7 Regressão	Motor, sessão, UI e revisão visual
8 Registro	Mudança e evidência vinculadas ao commit

Adicionar um jogo ou uma regra

Proposta de arquitetura: começar por engine sem DOM, fixtures pequenas e replay. Acrescentar adaptador de UI, rota com cancelamento, esquema de persistência e conteúdo mínimo certificado. Integrar categoria, cartão, regras, informação, ícone, testes e documentação. Essa sequência é recomendada; não existe no baseline um gerador automático de novos jogos.

Uma regra nova requer compatibilidade explícita: aceitar saves antigos, migrá-los com evidência ou arquivá-los sem reinterpretar a sessão. Nunca mudar o significado de uma seed antiga e afirmar que a partida é a mesma. Nunca corrigir um teste de regra apenas removendo a asserção que encontrou o defeito.

> Scripts com build, generate, audit ou record no nome podem escrever catálogos e relatórios. Antes de executá-los, leia o destino dos arquivos, trabalhe em uma cópia e revise o diff. Gerar documentação não autoriza publicar Site, criar release ou enviar comunicação oficial.

Fontes desta seção: Padrões observados nos diretórios dos jogos, migrações de campanha e scripts tests/*80-build.mjs.

12 Referências e limites

As referências abaixo são caminhos relativos à raiz do repositório e podem ser usadas em uma cópia independente. Para resolver divergências, priorize o módulo ativo e a evidência ligada à sua revisão; relatórios antigos permanecem úteis, mas não descrevem automaticamente o catálogo atual.

Assunto	Leitura de referência
Entrada e catálogo	dist/index.html; dist/app.mjs; dist/category-navigation.mjs
Motores originais	dist/light-world.mjs; dist/curves.mjs; dist/math.mjs
Coleção lógica	dist/geometry-games.mjs; dist/elo-rules.mjs; dist/symbol-games.mjs
UI e histórico	dist/collection-ui.mjs; dist/collection-state.mjs; dist/direct-input.mjs
Persistência	dist/*-progress.mjs; dist/answer-progress.mjs; UI dos jogos dedicados
Importação	dist/level-json-common.mjs; dist/original-json.mjs; dist/collection-json.mjs
Serviços	dist/repasse/service.mjs; dist/sintonia/service.mjs; dist/orbita/service.mjs; dist/cerco/service.mjs
Duelos	dist/desvio.mjs; dist/broto/engine.mjs; dist/cerco/engine.mjs
Catálogos atuais	dist/levels.mjs; dist/geometry-levels.mjs; dist/symbol-levels.mjs; campaign.mjs dos jogos
Complemento	Manual de QA e manutenção; Manual de game design e level design

O que não foi afirmado

• Não se afirma que a suíte global está aprovada. O baseline auditado tem falhas e a triagem em cópia local é uma atividade separada.

• Não se afirma validação de todo navegador, leitor de tela ou dispositivo, nem calibração de dificuldade com jogadores.

• Não se afirma que uma proposta de refatoração, runner, licença ou publicação foi implantada. Consulte o estado do repositório antes de usar comandos exclusivos da revisão local.

Data editorial: 1 de novembro de 2026. As explicações técnicas são síntese do código e dos registros disponíveis, com recomendações assinaladas. Leia os 11 manuais individuais para regras, exemplos e detalhes de cada jogo.